

5. Дерева рішень

Дерева рішень (Decision Trees) та ансамблеві методи машинного навчання - використовуються як для класифікації, так і для регресії. Дозволяють працювати зі складними нелінійними залежностями.

Перед початком роботи необхідно підключити бібліотеки:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer, fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor,
plot_tree
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier
from sklearn.metrics import accuracy_score

from xgboost import XGBClassifier
```

- `load_breast_cancer` - завантажує вбудований датасет Breast Cancer Wisconsin для задач класифікації (містить характеристики клітин пухлин молочної залози);
- `fetch_openml` - дозволяє завантажувати датасети з онлайн-платформи OpenML;
- `DecisionTreeClassifier` - реалізація дерева рішень для задач класифікації;
- `DecisionTreeRegressor` - реалізація дерева рішень для задач регресії;
- `plot_tree` - використовується для графічного відображення структури дерева рішень;
- `RandomForestClassifier` - ансамблевий алгоритм Random Forest, який будує багато дерев рішень та об'єднує їх результати;
- `GradientBoostingClassifier` - алгоритм ансамблевого навчання, у якому моделі навчаються послідовно, а кожна наступна намагається виправити помилки попередніх, що підвищує точність класифікації;
- `accuracy_score` - метрика для оцінки точності класифікації;

- **XGBClassifier** – алгоритм машинного навчання для класифікації табличних даних, який поєднує багато дерев рішень та використовує оптимізовані методи навчання, що забезпечують швидшу роботу й часто точніші результати.

Класифікація та візуалізація дерева рішень (Decision Tree)

Дерево рішень працює за принципом послідовного розділення даних на групи. На кожному кроці алгоритм вибирає ознаку та поріг поділу, які дозволяють найточніше відокремити один клас від іншого.

```
data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

dt_clf = DecisionTreeClassifier(criterion='entropy', max_depth=4,
                                random_state=42)
dt_clf.fit(X_train, y_train)

print("--- Класифікація (Decision Tree) ---")
print(f"Train Accuracy: {dt_clf.score(X_train, y_train):.3f}")
print(f"Test Accuracy: {dt_clf.score(X_test, y_test):.3f}")

plt.figure(figsize=(20, 10))
plot_tree(dt_clf, feature_names=data.feature_names,
          class_names=data.target_names, filled=True)
plt.title("Decision Tree Structure")
plt.show()
```

Функція `load_breast_cancer()` завантажує готовий набір даних для задач класифікації. Після цього `train_test_split()` розділяє дані на тренувальні та тестові.

Модель створюється через `DecisionTreeClassifier()`, тут: `criterion='entropy'` – використовується ентропія для оцінки якості розділення; `max_depth=4` – максимальна глибина дерева¹; `random_state=42` – фіксує випадковість побудови.

Після створення модель навчається через метод `fit()` – він аналізує ознаки та будує дерево рішень, поступово виконуючи розподіл (`split`) даних.

¹ Глибина дерева – спосіб контролювати складність моделі. Мала глибина робить її простішою, натомість велика може призвести до перенавчання.

Ентропія (Entropy) – це міра «хаосу» або змішаності даних. Якщо в одному вузлі дерева багато різних класів – ентропія висока, і навпаки.

Ентропія обчислюється за формулою:

$$Entropy = - \sum_{i=1}^n p_i \log_2(p_i)$$

де p_i – ймовірність належності до класу.

Альтернативою ентропії є індекс Джині (Gini Index). Він показує ймовірність помилкової класифікації випадкового об'єкта.

Індекс Джині обчислюється за формулою:

$$Gini = 1 - \sum_{i=1}^n p_i^2$$

Чим нижче значення Gini тим краще розділені класи. Тобто якщо Gini низький, то ентропія теж буде низькою. Різниця в способі обчислення і чутливості (Gini трохи простіше і швидше), але на практиці обидва майже завжди дають схожі результати.

Під час побудови дерева алгоритм оцінює, наскільки добре новий розподіл зменшує невизначеність². Для цього використовується формула інформаційного приросту (Information Gain):

$$Information\ Gain = Entropy(parent) - \sum_k \frac{|D_k|}{|D|} Entropy(D_k)$$

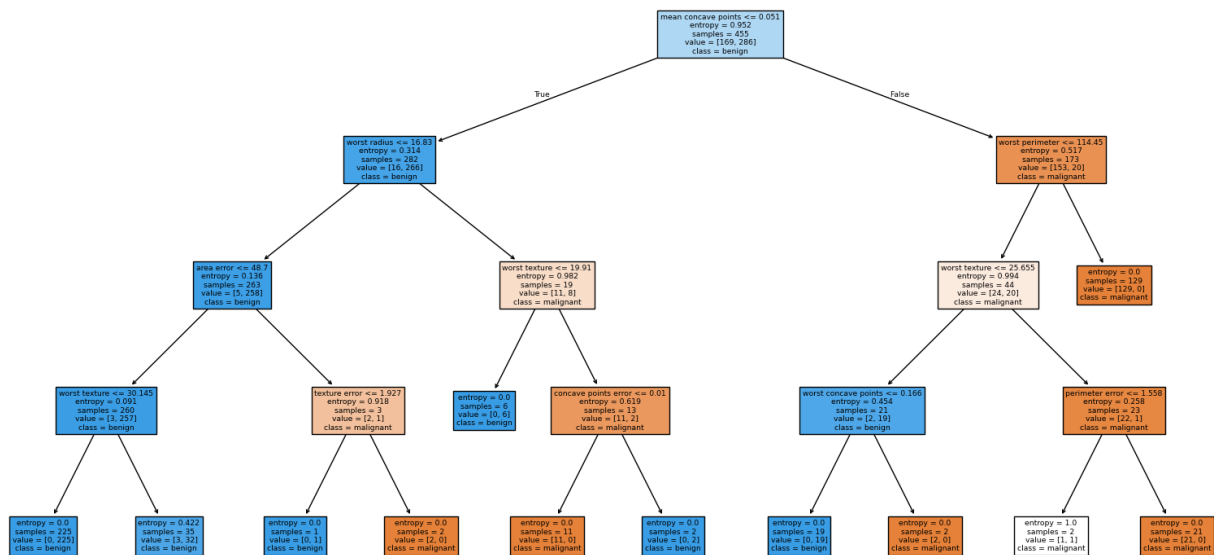
Інформаційний приріст оцінює, наскільки зменшився хаос після розподілу і обирає той розподіл, який дає найбільший приріст інформації.

Для відображення структури дерева використовується `plot_tree()`, тут: кожен вузол містить умову розподілу, показується кількість об'єктів та відображається домінуючий клас.

Параметр `filled=True` зафарбовує вузли відповідно до класу та рівня «чистоти» вузла³.

² Невизначеність – це міра того, наскільки змішані класи в одному вузлі. Якщо класи розподілені порівну (50/50), то вона дуже висока, а ентропія в такому вузлі максимальна.

³ «Чистота вузла» – чим менше невизначеність, тим чистіший вузол.



Тобто ось дерево рішень - ми можемо чітко відстежити, чому модель вирішила виконати саме такий розподіл. Вузли з класом «доброякісні» зафарбовані в синій, «злроякісні» в помаранчевий. Насиченість цих кольорів залежить від рівня ентропії, наприклад там де вона низька (0.0) колір дуже насичений, а там де висока (1.0) вузол взагалі білий (втратив колір).

Випадковий ліс (Random Forest)

Одне дерево рішень може легко перенавчитись, особливо якщо його глибина велика. Для вирішення цієї проблеми використовують ансамблеві методи - підхід, при якому замість однієї моделі працює одразу багато.

```
print("--- Ансамблевi методи ---")

rf_clf = RandomForestClassifier(n_estimators=100, max_depth=4,
                               random_state=42)
rf_clf.fit(X_train, y_train)
print(f"Random Forest Accuracy: {rf_clf.score(X_test, y_test):.3f}")

gb_clf = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1,
                                    max_depth=3, random_state=42)
gb_clf.fit(X_train, y_train)
print(f"Gradient Boosting Accuracy: {gb_clf.score(X_test, y_test):.3f}")
```

`RandomForestClassifier()` створює ансамбль дерев рішень, тут: `n_estimators=100` - кількість дерев у лісі; `max_depth=4` - максимальна глибина кожного дерева.

Випадковий ліс використовує метод Bagging (Bootstrap Aggregating) – це метод, де кілька моделей навчаються на різних випадкових підвбірках даних і їхні результати об'єднуються для більш стабільного прогнозу.

Після навчання всі дерева роблять власний прогноз. Фінальним результатом для класифікації буде більшість голосів, для регресії – середнє значення прогнозів.

Схематично це можна подати так:

$$Prediction = \frac{1}{N} \sum_{i=1}^N Tree_i(x)$$

де:

- N – кількість дерев,
- $Tree_i(x)$ – прогноз окремого дерева.

На відміну від одного дерева рішень, випадковий ліс значно стабільніший, менш чутливий до шуму та майже не перенавчається.

Ще одним ансамблевим методом є градієнтне підсилення (Gradient Boosting) – це метод, у якому модель будується послідовно. Кожна нова намагається виправити помилки попередніх, щоб поступово підвищити точність прогнозу.

`GradientBoostingClassifier()` створює градієнтне підсилення, тут: `n_estimators=100` – кількість дерев; `learning_rate=0.1` – швидкість навчання⁴; `max_depth=3` – максимальна глибина дерев.

Градієнтне підсилення зазвичай дає вищу точність, але працює повільніше (послідовна побудова дерев вимагає більше часу, у випадковому лісі вони будуються паралельно), також більш чутливий до перенавчання та сильніше залежить від параметрів моделі.

⁴ Швидкість навчання впливає на те, наскільки великими кроками модель вчиться виправляти свої помилки. При малих значеннях модель вчиться повільніше але працює стабільніше і точніше.

Екстремальне підсилення (XGBoost)

XGBoost (Extreme Gradient Boosting) - один із найпопулярніших алгоритмів для роботи з табличними даними. Він є покращеною та оптимізованою версією Gradient Boosting, поєднує високу точність, швидкість та механізми боротьби з перенавчанням.

```
xgb_clf = XGBClassifier(n_estimators=100, learning_rate=0.05,
                        verbosity=0, use_label_encoder=False)
xgb_clf.fit(X_train, y_train)

xgb_pred = xgb_clf.predict(X_test)
print(f"XGBoost Accuracy: {accuracy_score(y_test, xgb_pred):.3f}")

plt.figure(figsize=(10, 6))
feat_importances = pd.Series(xgb_clf.feature_importances_,
                             index=data.feature_names)
feat_importances.nlargest(10).plot(kind='barh')
plt.title("Top 10 Important Features (XGBoost)")
plt.show()
```

XGBClassifier() створює модель екстремального підсилення, тут: `n_estimators=100` - кількість дерев; `learning_rate=0.05` - швидкість навчання; `verbosity=0` - вимикає службові повідомлення; `use_label_encoder=False` - відключає старий механізм кодування міток.

На відміну від випадкового лісу дерева будуються послідовно, кожне нове дерево намагається виправити помилки попередніх моделей. Тобто це покращена реалізація градієнтного підсилення дерев.

Однією з головних переваг XGBoost є вбудована регуляризація. Алгоритм підкримує L1-регуляризацію (Lasso) та L2-регуляризацію (Ridge). Регуляризація додає обмеження до функції втрат:

$$Loss = Training Loss + \lambda \|w\|^2 + \alpha \|w\|$$

де:

- $\|w\|^2$ - L2-регуляризація,
- $\|w\|$ - L1-регуляризація,
- λ та α - коефіцієнти обмеження.

Завдяки цьому модель менше перенавчається, покращується узагальнення та зростає стабільність до нових даних.

Також XGBoost оптимізований для роботи з великими наборами даних. Алгоритм підтримує паралельні обчислення, оптимізовану роботу з пам'яттю та швидке побудування дерев. Завдяки цьому модель працює значно швидше за класичний Gradient Boosting та добре масштабується на великих датасетах.

Ще одна перевага XGBoost - автоматична обробка пропущених значень (missing values). Алгоритм може самостійно визначити оптимальний напрямок розподілу для NaN, що дозволяє йому працювати без попередньо заповнених пропущених значень. Це значно спрощує підготовку даних.

Для оцінки точності виконастовується `accuracy_score()` - метод обчислює частку правильних прогнозів моделі.

Також XGBoost дозволяє оцінити, які ознаки найбільше впливають на прогноз. Для цього використовується `xgb_clf.feature_importances_`.

Тут створюється серія, де значеннями є важливість ознак, а індексами - назви ознак:

```
feat_importances = pd.Series(xgb_clf.feature_importances_,
                             index=data.feature_names)
feat_importances.nlargest(10).plot(kind='barh')
```

Після цього: `nlargest(10)` - вибирає 10 найважливіших ознак; `plot(kind='barh')` - будує горизонтальний bar chart.

Регресія за допомогою дерев

Дерева рішень можуть використовуватись не лише для класифікації, а й для задач регресії, тобто прогнозування числового значення (ціни, температури тощо).

```
housing = fetch_openml(name="house_prices", as_frame=True, parser='auto')
X_reg = housing.data.select_dtypes(include=[np.number]).fillna(0)
y_reg = housing.target

X_r_train, X_r_test, y_r_train, y_r_test = train_test_split(X_reg, y_reg,
                                                            test_size=0.2, random_state=42)
dt_reg = DecisionTreeRegressor(max_depth=5, random_state=42)
dt_reg.fit(X_r_train, y_r_train)
print("--- Перспекція (House Prices) ---")
print(f"R^2 Score on Test: {dt_reg.score(X_r_test, y_r_test):.3f}")
```

Функція `fetch_openml()` завантажує набір даних House Prices з платформи OpenML, тут: `as_frame=True` - дані повертаються у вигляді DataFrame; `parser='auto'` - автоматичний вибір способу обробки даних.

Для моделі вибираються лише числові ознаки за допомогою `select_dtypes()`. Після цього пропущені значення замінюються на нуль.

Модель створюється через `DecisionTreeRegressor()`, тут: `max_depth=5` - обмежує глибину дерева; `random_state=42` - фіксує випадковість побудови.

Основна відмінність між класифікацією (`DecisionTreeClassifier`) та регресією (`DecisionTreeRegressor`) - в листках дерева зберігається не клас, а середнє значення цільової змінної.

Для регресії замість Gini або Entropy використовується MSE - середньоквадратична помилка. MSE обчислюється як:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

де:

- y_i - реальне значення,
- $\hat{y}_i$ - прогноз моделі.

Дерево намагається знайти такі розподіли, які мінімізують середньоквадратичну помилку у вузлах. Таким чином, чим менше MSE - тим точніше прогноз моделі.

Для оцінки якості регресії використовується `score()` - метод повертає коефіцієнт детермінації R^2 .